



Proximal Policy Optimization

Deep Reinforcement Learning
Bawornsak Sakulkueakulsuk

Kullback–Leibler Divergence

In Reinforcement Learning, we deal with a lot of distribution.

- Policy
- Value
- Reward

To solve RL, we need to “improve” some distributions

- We need to improve a policy. But how do we compare the policy?

Kullback–Leibler Divergence

Let's simplify. Given 2 coins: 1 fair, 1 unfair,

$$\text{Coin 1: } \begin{cases} 0.5 & \text{heads} \\ 0.5 & \text{tails} \end{cases}$$

$$\text{Coin 2: } \begin{cases} p_1 & \text{heads} \\ p_2 & \text{tails} \end{cases}$$

If we throw Coin 1 10 times, what can we expect?

Kullback–Leibler Divergence

Let's simplify. Given 2 coins: 1 fair, 1 unfair,

$$\text{Coin 1: } \begin{cases} 0.5 & \text{heads} \\ 0.5 & \text{tails} \end{cases}$$

$$\text{Coin 2: } \begin{cases} p_1 & \text{heads} \\ p_2 & \text{tails} \end{cases}$$

If we throw Coin 1 10 times, what can we expect?

H H T T H T T T H H

Kullback–Leibler Divergence

Let's simplify. Given 2 coins: 1 fair, 1 unfair,

$$\text{Coin 1: } \begin{cases} 0.5 & \text{heads} \\ 0.5 & \text{tails} \end{cases}$$

$$\text{Coin 2: } \begin{cases} p_1 & \text{heads} \\ p_2 & \text{tails} \end{cases}$$

If we throw Coin 1 10 times, what can we expect?

H H T T H T T T H H

If $p_1 = 0.55$ and we throw Coin 2 10 times, what can we expect?

Kullback–Leibler Divergence

Let's simplify. Given 2 coins: 1 fair, 1 unfair,

$$\text{Coin 1: } \begin{cases} 0.5 & \text{heads} \\ 0.5 & \text{tails} \end{cases}$$

$$\text{Coin 2: } \begin{cases} p_1 & \text{heads} \\ p_2 & \text{tails} \end{cases}$$

If we throw Coin 1 10 times, what can we expect?

H H T T H T T T H H

If $p_1 = 0.55$ and we throw Coin 2 10 times, what can we expect?

H H **H** T H T T T H H

Kullback–Leibler Divergence

Let's simplify. Given 2 coins: 1 fair, 1 unfair,

$$\text{Coin 1: } \begin{cases} 0.5 & \text{heads} \\ 0.5 & \text{tails} \end{cases}$$

$$\text{Coin 2: } \begin{cases} p_1 & \text{heads} \\ p_2 & \text{tails} \end{cases}$$

If we throw Coin 1 10 times, what can we expect?

H H T T H T T T H H

If $p_1 = 0.55$ and we throw Coin 2 10 times, what can we expect?

H H **H** T H T T T H H

If $p_1 = 0.95$ and we throw Coin 2 10 times, what can we expect?

Kullback–Leibler Divergence

Let's simplify. Given 2 coins: 1 fair, 1 unfair,

$$\text{Coin 1: } \begin{cases} 0.5 & \text{heads} \\ 0.5 & \text{tails} \end{cases}$$

$$\text{Coin 2: } \begin{cases} p_1 & \text{heads} \\ p_2 & \text{tails} \end{cases}$$

If we throw Coin 1 10 times, what can we expect?

H H T T H T T T H H

If $p_1 = 0.55$ and we throw Coin 2 10 times, what can we expect?

H H **H** T H T T T H H

If $p_1 = 0.95$ and we throw Coin 2 10 times, what can we expect?

H H **H** T H **H H** T H H

Kullback–Leibler Divergence

Let's simplify. Given 2 coins: 1 fair, 1 unfair,

$$\text{Coin 1: } \begin{cases} 0.5 & \text{heads} \\ 0.5 & \text{tails} \end{cases}$$

$$\text{Coin 2: } \begin{cases} p_1 & \text{heads} \\ p_2 & \text{tails} \end{cases}$$

If we throw Coin 1 10 times, what can we expect?

H H T T H T T T H H

If $p_1 = 0.55$ and we throw Coin 2 10 times, what can we expect?

H H **H** T H T T T H H

If $p_1 = 0.95$ and we throw Coin 2 10 times, what can we expect?

H H **H** T H **H H** T H H

What distribution is closer to a fair coin?

Kullback–Leibler Divergence

Let's consider

Coin 1: $\begin{cases} p_1 & \text{heads} \\ p_2 & \text{tails} \end{cases}$

Coin 2: $\begin{cases} q_1 & \text{heads} \\ q_2 & \text{tails} \end{cases}$

H H T T H T T T H H

$P(\text{Observation}|\text{Coin1}) =$

$P(\text{Observation}|\text{Coin2}) =$

Kullback–Leibler Divergence

Let's consider

Coin 1: $\begin{cases} p_1 & \text{heads} \\ p_2 & \text{tails} \end{cases}$

Coin 2: $\begin{cases} q_1 & \text{heads} \\ q_2 & \text{tails} \end{cases}$

H H T T H T T T H H

$$P(\text{Observation}|\text{Coin1}) = p_1^{N_H} p_2^{N_T}$$

$$P(\text{Observation}|\text{Coin2}) = q_1^{N_H} q_2^{N_T}$$

Kullback–Leibler Divergence

To compare two distributions, we can divide one event by the other.

$$\frac{P(\text{Observation}|\text{Coin1})}{P(\text{Observation}|\text{Coin2})} = \frac{p_1^{N_H} p_2^{N_T}}{q_1^{N_H} q_2^{N_T}}$$

And modify the above expression by taking logarithm and powering by $\frac{1}{N}$.

Kullback–Leibler Divergence

$$\begin{aligned}
 \log \left(\frac{p_1^{N_H} p_2^{N_T}}{q_1^{N_H} q_2^{N_T}} \right)^{\frac{1}{N}} &= \frac{1}{N} \log \left(\frac{p_1^{N_H} p_2^{N_T}}{q_1^{N_H} q_2^{N_T}} \right) \\
 &= \frac{1}{N} \log p_1^{N_H} + \frac{1}{N} \log p_2^{N_T} - \frac{1}{N} \log q_1^{N_H} - \frac{1}{N} \log q_2^{N_T} \\
 &= \frac{N_H}{N} \log p_1 + \frac{N_T}{N} \log p_2 - \frac{N_H}{N} \log q_1 - \frac{N_T}{N} \log q_2 \\
 &= p_1 \log p_1 + p_2 \log p_2 - p_1 \log q_1 - p_2 \log q_2 \\
 &= p_1 \log \frac{p_1}{q_1} + p_2 \log \frac{p_2}{q_2}
 \end{aligned}$$

In general form,

$$D_{KL}(P||Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

Kullback–Leibler Divergence

$$D_{KL}(P||Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

KL Divergence is a type of statistical distance to measure differences of a probability distribution Q when comparing to a true probability distribution P

- We have an actual distribution P
- We have a model distribution Q
- We want to know the expected excess surprise

Trust Region Policy Optimization

Policy Gradient Review

In policy gradient, we compute and update the policy by

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha R_{t+1} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}_t}(A_t | S_t)$$

This approach suggests that we can update $\boldsymbol{\theta}$ toward the direction that increase the gradient the most.

But how much should we update? Can we really go all the way?



Trust Region Policy Optimization

Key ideas:

- We want to maximize the update step
- However, the update must not be too far from the old policy
- Hence, the “trust region”

Analysis:

- Vanilla policy gradients keep new policies close to old policies in the parameter space
- However, this can impact performance by a lot
- TRPO aims to tackle this problem

Advantage Function

To evaluate policy, we introduce an **advantage** function.

$$\text{Let } Q_{\pi}(s_t, a_t) = \mathbb{E}_{s_{t+1}, a_{t+1}, \dots} [\sum_l^{\infty} \gamma^l r(s_{t+l})]$$

$$V_{\pi}(s_t) = \mathbb{E}_{a_t, s_{t+1}, \dots} [\sum_l^{\infty} \gamma^l r(s_{t+l})]$$

Then the advantage function is

$$A_{\pi}(s, a) = Q_{\pi}(s, a) - V_{\pi}(s), \text{ where}$$

$$a_t \sim \pi(a_t | s_t),$$

$$s_{t+1} \sim P(s_{t+1} | s_t, a_t) \text{ for } t \geq 0$$

Surrogate Advantage

The surrogate advantage measures how policy π_{θ} performs relatively to the old policy $\pi_{\theta_{old}}$ using data from the old policy.

$$\mathcal{L}(\theta_{old}, \theta) = \mathbb{E}_{s,a \sim \pi_{\theta_{old}}} \left[\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)} A_{\pi_{\theta_{old}}}(s, a) \right]$$

Trust Region Policy Optimization

Trust Region Policy Optimization updates the parameters of the policy by

$$\begin{aligned} \boldsymbol{\theta}_{\text{new}} &= \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \mathcal{L}(\boldsymbol{\theta}_{\text{old}}, \boldsymbol{\theta}) \\ \text{s.t. } \bar{D}_{KL}(\boldsymbol{\theta}_{\text{old}} || \boldsymbol{\theta}) &\leq \delta \end{aligned}$$

Given

$$\mathcal{L}(\boldsymbol{\theta}_{\text{old}}, \boldsymbol{\theta}) = \mathbb{E}_{s, a \sim \pi_{\boldsymbol{\theta}_{\text{old}}}} \left[\frac{\pi_{\boldsymbol{\theta}}(a|s)}{\pi_{\boldsymbol{\theta}_{\text{old}}}(a|s)} A_{\pi_{\boldsymbol{\theta}_{\text{old}}}}(s, a) \right]$$

$$\bar{D}_{KL}(\boldsymbol{\theta}_{\text{old}} || \boldsymbol{\theta}) = \mathbb{E}_{s \sim \pi_{\boldsymbol{\theta}_{\text{old}}}} [D_{KL}(\pi_{\boldsymbol{\theta}_{\text{old}}}(\cdot | s) || \pi_{\boldsymbol{\theta}}(\cdot | s))]$$

Trust Region Policy Optimization

To update and solve TRPO, there are a lot of calculation and optimization. We won't cover them here.

Generally, it requires a lot of complex calculation.

However, it can be modified to simplify the process through **Proximal Policy Optimization**.

Proximal Policy Optimization

Proximal Policy Optimization (PPO) aims to simplify TRPO complex calculation while maintaining relatively close or better performance.

PPO still aims to take the biggest step to update its policy, but not too big to make the performance worse.

There are two primary variants of PPO.

- PPO-Penalty: use KL divergence as a penalty instead of a hard constraint
- PPO-Clip: No KL Term. No constraints. Clipping the update instead.

Modified Trust Region Method

Instead of using KL Divergence as a hard constrain,

$$\begin{aligned} \boldsymbol{\theta}_{\text{new}} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \mathcal{L}(\boldsymbol{\theta}_{\text{old}}, \boldsymbol{\theta}) \\ \text{s.t. } \bar{D}_{KL}(\boldsymbol{\theta}_{\text{old}} || \boldsymbol{\theta}) \leq \delta \end{aligned}$$

Given

$$\begin{aligned} \mathcal{L}(\boldsymbol{\theta}_{\text{old}}, \boldsymbol{\theta}) &= \mathbb{E}_{s, a \sim \pi_{\boldsymbol{\theta}_{\text{old}}}} \left[\frac{\pi_{\boldsymbol{\theta}}(a|s)}{\pi_{\boldsymbol{\theta}_{\text{old}}}(a|s)} A_{\pi_{\boldsymbol{\theta}_{\text{old}}}}(s, a) \right] \\ \bar{D}_{KL}(\boldsymbol{\theta}_{\text{old}} || \boldsymbol{\theta}) &= \mathbb{E}_{s \sim \pi_{\boldsymbol{\theta}_{\text{old}}}} [D_{KL}(\pi_{\boldsymbol{\theta}_{\text{old}}}(\cdot | s) || \pi_{\boldsymbol{\theta}}(\cdot | s))] \end{aligned}$$

PPO modifies the constrain into a penalty instead

$$\max_{\boldsymbol{\theta}} \mathbb{E}_t \left[\frac{\pi_{\boldsymbol{\theta}}(a|s)}{\pi_{\boldsymbol{\theta}_{\text{old}}}(a|s)} A_{\pi_{\boldsymbol{\theta}_{\text{old}}}}(s, a) - \beta D_{KL}(\pi_{\boldsymbol{\theta}_{\text{old}}}(\cdot | s) || \pi_{\boldsymbol{\theta}}(\cdot | s)) \right], \quad \beta \in \mathbb{R}$$

Adaptive KL Penalty

One PPO variant use the modified trust region as an objective function. The variable β can be fixed or adaptive. For the adaptive KL Penalty, the objective function is

$$\mathcal{L}^{KL PEN}(\boldsymbol{\theta}) = \mathbb{E}_t \left[\frac{\pi_{\boldsymbol{\theta}}(a|s)}{\pi_{\boldsymbol{\theta}_{old}}(a|s)} A_{\pi_{\boldsymbol{\theta}_{old}}}(s, a) - \beta D_{KL}(\pi_{\boldsymbol{\theta}_{old}}(\cdot | s) || \pi_{\boldsymbol{\theta}}(\cdot | s)) \right]$$

Then compute $d = \mathbb{E}[D_{KL}(\pi_{\boldsymbol{\theta}_{old}}(\cdot | s) || \pi_{\boldsymbol{\theta}}(\cdot | s))]$

- If $d < \frac{d_{targ}}{1.5}$, $\beta \leftarrow \frac{\beta}{2}$
- If $d > d_{targ}$, $\beta \leftarrow \frac{\beta}{2}$

Algorithm 4 PPO with Adaptive KL Penalty

Input: initial policy parameters θ_0 , initial KL penalty β_0 , target KL-divergence δ

for $k = 0, 1, 2, \dots$ **do**

Collect set of partial trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Compute policy update

$$\theta_{k+1} = \arg \max_{\theta} \mathcal{L}_{\theta_k}(\theta) - \beta_k \bar{D}_{KL}(\theta || \theta_k)$$

by taking K steps of minibatch SGD (via Adam)

if $\bar{D}_{KL}(\theta_{k+1} || \theta_k) \geq 1.5\delta$ **then**

$$\beta_{k+1} = 2\beta_k$$

else if $\bar{D}_{KL}(\theta_{k+1} || \theta_k) \leq \delta/1.5$ **then**

$$\beta_{k+1} = \beta_k/2$$

end if

end for

Clipped Surrogate Objective

Another PPO variant bounds the update by a hyperparameter, clipping the update using a lower bound to ensure a safe update.

Let

$$r_t(\boldsymbol{\theta}) = \frac{\pi_{\boldsymbol{\theta}}(a|s)}{\pi_{\boldsymbol{\theta}_{old}}(a|s)}$$

Thus

$$r(\boldsymbol{\theta}_{old}) = 1$$

PPO Surrogate Objective Comparison

So, the TRPO maximizes a following surrogate objective

$$\mathcal{L}^{CPI}(\boldsymbol{\theta}) = \mathbb{E}_t \left[\frac{\pi_{\boldsymbol{\theta}}(a_t | s_t)}{\pi_{\boldsymbol{\theta}_{old}}(a_t | s_t)} A_t \right] = \mathbb{E}_t [r_t(\boldsymbol{\theta}) A_t]$$

We then clipped the update using a following equation

$$\mathcal{L}^{CLIP}(\boldsymbol{\theta}) = \mathbb{E}_t [\min(r_t(\boldsymbol{\theta}) A_t, \text{clip}(r_t(\boldsymbol{\theta}), 1 - \epsilon, 1 + \epsilon) A_t)]$$

PPO Surrogate Objective Comparison

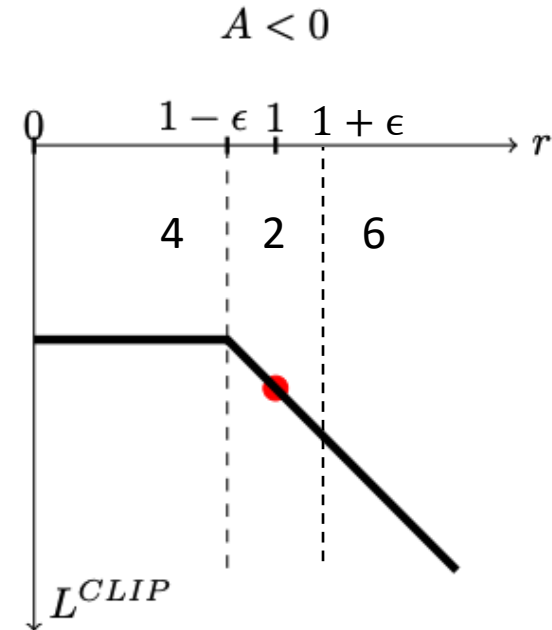
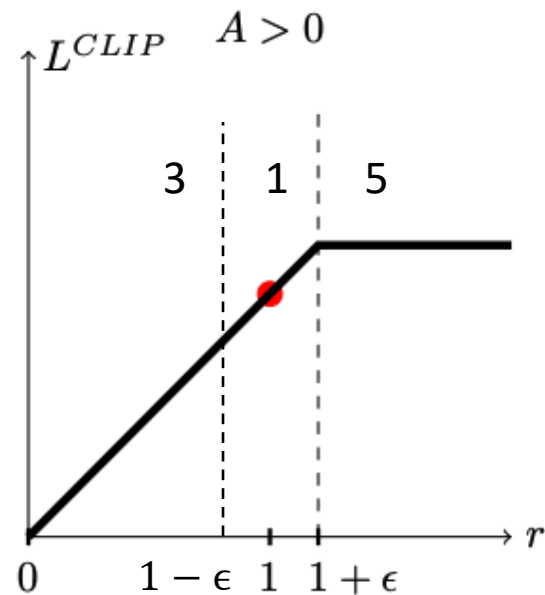
$$\mathcal{L}^{CLIP}(\boldsymbol{\theta}) = \mathbb{E}_t[\min(r_t(\boldsymbol{\theta})A_t, \text{clip}(r_t(\boldsymbol{\theta}), 1 - \epsilon, 1 + \epsilon)A_t)]$$

This term basically bound the updated policy between $[1 - \epsilon, 1 + \epsilon]$, for example, $\epsilon = 0.2$.

It forces the update to not go outside our “zone”. Compared to Policy Gradient, this approach forces our agent to keep improving in a safe amount and direction.

$$\mathcal{L}^{CLIP}(\boldsymbol{\theta}) = \mathbb{E}_t[\min(r_t(\boldsymbol{\theta})A_t, \text{clip}(r_t(\boldsymbol{\theta}), 1 - \epsilon, 1 + \epsilon)A_t)]$$

	$p_t(\theta) > 0$	A_t	Return Value of $\min$	Objective is Clipped	Sign of Objective	Gradient
1	$p_t(\theta) \in [1 - \epsilon, 1 + \epsilon]$	+	$p_t(\theta)A_t$	no	+	✓
2	$p_t(\theta) \in [1 - \epsilon, 1 + \epsilon]$	−	$p_t(\theta)A_t$	no	−	✓
3	$p_t(\theta) < 1 - \epsilon$	+	$p_t(\theta)A_t$	no	+	✓
4	$p_t(\theta) < 1 - \epsilon$	−	$(1 - \epsilon)A_t$	yes	−	0
5	$p_t(\theta) > 1 + \epsilon$	+	$(1 + \epsilon)A_t$	yes	+	0
6	$p_t(\theta) > 1 + \epsilon$	−	$p_t(\theta)A_t$	no	−	✓



Algorithm 5 PPO with Clipped Objective

Input: initial policy parameters θ_0 , clipping threshold ϵ

for $k = 0, 1, 2, \dots$ **do**

Collect set of partial trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Compute policy update

$$\theta_{k+1} = \arg \max_{\theta} \mathcal{L}_{\theta_k}^{CLIP}(\theta)$$

by taking K steps of minibatch SGD (via Adam), where

$$\mathcal{L}_{\theta_k}^{CLIP}(\theta) = \mathbb{E}_{\tau \sim \pi_k} \left[\sum_{t=0}^T \left[\min(r_t(\theta) \hat{A}_t^{\pi_k}, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^{\pi_k}) \right] \right]$$

end for

Curriculum Learning

A powerful approach in Machine Learning and Reinforcement Learning to teach an agent to learn faster and more efficient by increasing the difficulty or complexity of task along the learning.

- We can start with simple tasks, commands, or obstacles
- Once an agent can perform the task effectively, we increase the difficulty of the task or complexity of the commands.
- This way ensures the growth of learning process for the agent.